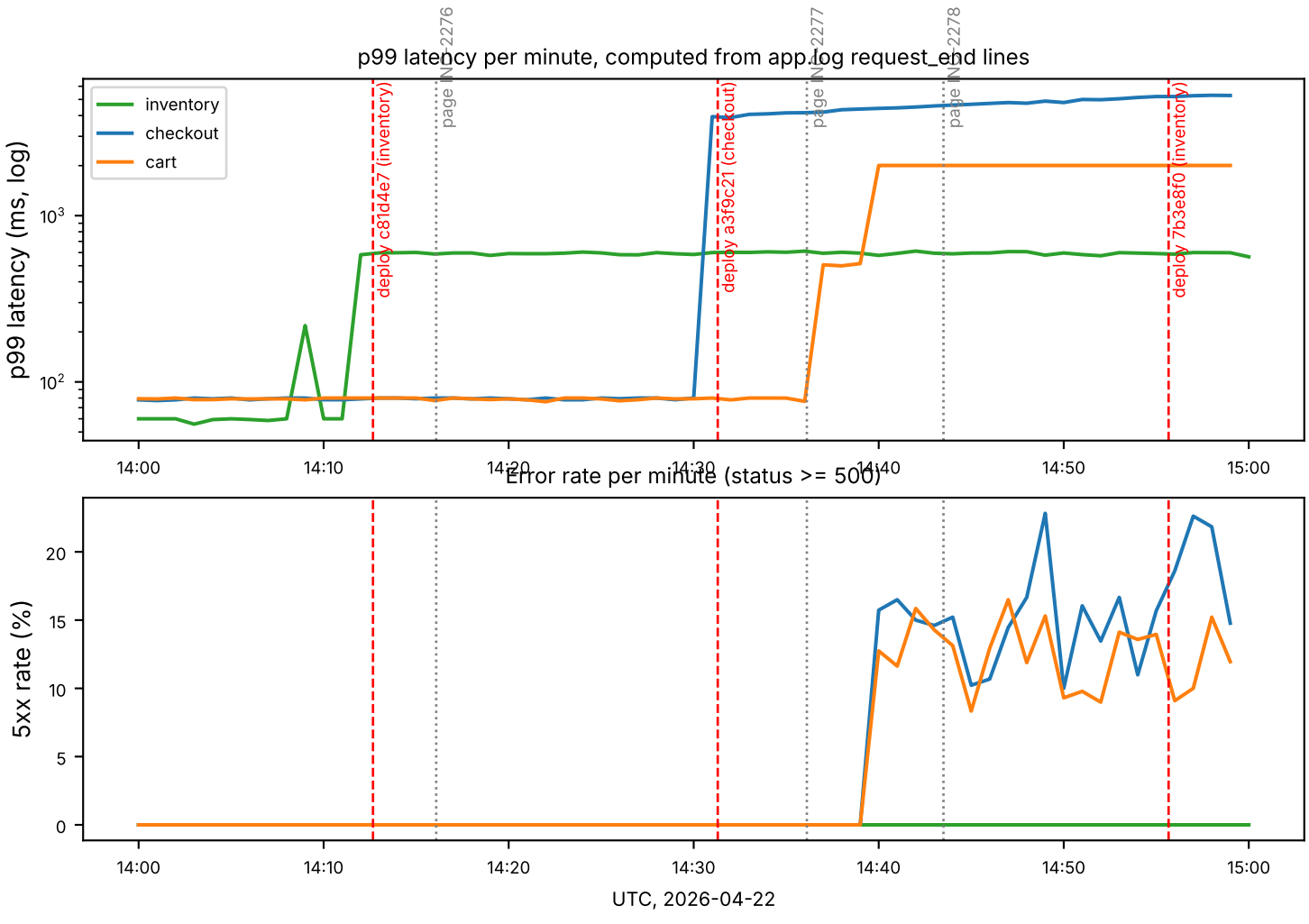
Postmortem: inventory latency, checkout latency and cart errors (2026-04-22, two incidents)

Two unrelated deploys degraded three services in the same hour. Commit c81d4e7 removed inventory's SKU availability cache at 14:12:40, raising inventory p99 from ~44 ms to ~600 ms (INC-2276). Commit a3f9c21 introduced an N+1 query in checkout's order summary at 14:31:18, saturating the shared db-primary: checkout p99 reached 3.6 s with up to 21% PoolTimeout 500s (INC-2277), and cart, which shares db-primary, returned up to 13.7% DBTimeout 503s (INC-2278). Neither deploy had been rolled back at 15:00:00, the end of the data.

Page	Service	Signal / fired	Root cause	Role
INC-2276	inventory	p99 > 300 ms, 14:16:05	Commit c81d4e7 removed @cached from get_sku_availability	Incident A (origin)
INC-2277	checkout	p99 > 500 ms, 14:36:07	Commit a3f9c21: N+1 order_items query in build_summary	Incident B (origin)
INC-2278	cart	error rate > 5%, 14:43:30	Same as INC-2277: shared db-primary saturated	Symptom of Incident B



Incident A: INC-2276, inventory p99 latency (commit c81d4e7)

Root cause

Commit c81d4e7 ("simplify SKU availability lookup ahead of cache-key migration") replaced @cached(key="sku:{sku}", ttl=300) on get_sku_availability in inventory/availability.py with a TODO to restore it after the sku:<region>:<id> key migration. Every availability check now runs an uncached SUM/GROUP BY over sku_stock on db-inventory. The same diff removed the cache_miss debug log, so misses stopped appearing in logs.

Timeline (UTC)

- 14:12:39 last cache_hit line in app.log
- 14:12:40 deploy_complete c81d4e7 (inventory, rolling)
- 14:13:00 cache_hit_rate 0.99 → 0.00; inventory p99 44 → 607 ms; db-inventory qps ~1.3 → 11-14
- 14:16:05 INC-2276 fires (p99 > 300 ms for 3m)
- 14:55:40 deploy 7b3e8f0 (inventory, SKU validation on reserve). Not a revert; no effect on latency
- 15:00:00 not recovered: p99 399 ms, cache_hit_rate 0.00

Evidence

- 2,827 'SELECT sku, SUM(qty) FROM sku_stock WHERE sku = ? GROUP BY warehouse' log lines after the deploy, 1 before. Exactly one per request, 220-560 ms each (median 388 ms).
- Inventory p99 32-45 ms before 14:12, 383-620 ms every minute after. db-inventory CPU 0.07 → ~0.40.
- No errors: all 3,654 inventory responses were 200; error_rate stayed 0.1-0.4%.

Ruled out

- Cold cache after restart (as in INC-2190): hit rate stayed at 0 for 47 minutes instead of refilling; there is no cache left to warm.
- Other services: db-inventory is used only by inventory; inventory metrics did not move at the 14:31:18 checkout deploy.
- Commit 7b3e8f0: deployed 43 minutes after onset; adds validation only.

Fix

Revert c81d4e7 to restore the cache. Redo the key migration with dual-read/dual-write so the cache stays in place throughout, and alert on inventory cache_hit_rate.

Incident B: INC-2277 checkout latency + INC-2278 cart errors (commit a3f9c21)

Root cause

Commit a3f9c21 ("refactor order summary builder for readability") replaced the batched OrderItem.objects.filter(order_id__in=order_ids) in CheckoutSummaryHandler.build_summary with a loop running OrderItem.objects.filter(order_id=order.id) per order (N+1). db-primary load rose ~11x and its CPU hit 0.99. Checkout's 50-connection pool filled (PoolTimeout 500s). Cart shares db-primary, so its single cart_items query per request exceeded its 2,000 ms timeout (DBTimeout 503s). INC-2278 is a symptom of this incident.

Timeline (UTC)

- 14:31:18 deploy_complete a3f9c21 (checkout); first per-order order_items query at 14:31:18.442
- 14:32:00 checkout p99 69 → 347 ms; db-primary qps 42 → 212 (446 at 14:33); CPU 0.22 → 0.45
- 14:36:07 INC-2277 fires (checkout p99 2,136 ms)
- 14:37:00 checkout pool 1.0; db-primary CPU 0.99; cart pool 0.14 → 0.31; first cart slow-query WARN
- 14:40:06 first 'DB connection pool exhausted (active=50/50)' and PoolTimeout 500; checkout p99 peaks 3,640 ms
- 14:40:08 first cart DBTimeout 503; cart error rate 0.1% → 13.7%
- 14:43:30 INC-2278 fires (cart error rate > 5%)
- 15:00:00 not recovered: checkout p99 3,356 ms, checkout pool 1.0, cart error rate 9.4%, db-primary 470 qps

Evidence

- 30,892 checkout 'SELECT * FROM order_items WHERE order_id = ?' lines, none before the deploy; 8-16 per request (median 12) across 2,586 requests.
- 281 checkout PoolTimeout 500s; checkout error rate 8-21% after 14:40.
- 223 cart 'db query timeout after 2000ms' on cart_items, each paired with a 503 DBTimeout (~2,004 ms); 831 cart slow-query WARNs, none before 14:37.
- Cart load unchanged (~900 requests / 10 min all hour); every cart request issues exactly one query.

Ruled out

- Cart code: last cart deploy e21b6c9 at 09:14:05 (lodash bump), five clean hours before onset.
- Inventory c81d4e7 / INC-2276: separate database; checkout p99 flat (55-78 ms) and db-primary flat 14:12-14:31.
- Auth (9f1e2d4, 11:02:33) and payment provider: no related errors in app.log; cart does not depend on auth.

Fix

Revert a3f9c21 to restore the batched order_id__in query. Verify db-primary back to ~42 qps, checkout pool < 0.2, checkout p99 ~65 ms, cart errors ~0.1%. Cart needs no change.

Prior incident

Same N+1 class as INC-2101. Its open follow-up, a CI test capping query count per request in checkout handlers, would have caught a3f9c21 before deploy.

Action items

#	Action	Owner team	Priority
1	Revert commit a3f9c21; verify db-primary qps, checkout pool, checkout p99 and cart error rate recover	Checkout	P0
2	Revert commit c81d4e7; verify cache_hit_rate returns to ~0.99 and inventory p99 to ~40 ms	Inventory	P0
3	Ship the query-count-per-request CI test for checkout handlers (open since INC-2101)	Checkout	P1
4	Redo the sku cache-key migration with dual-read/dual-write; never remove the cache	Inventory	P1
5	Alert on inventory cache_hit_rate dropping (e.g. < 0.5 for 2m)	SRE	P1
6	Per-service connection caps on db-primary so one service cannot starve another	SRE / DB	P2
7	Keep the cache_miss debug log (or a miss counter) so cache behaviour is visible in logs	Inventory	P2